

ACADEMIC TASK-2

CSE-316

(OPERATING SYSTEM)

COMPUTER SCIENCE AND ENGINEERING

PROJECT NAME:- ADVANCED DISK SIMULATOR

Submitted by:

Name: Adarsh kamal

Registration number: 12306714

Roll No.: 39

Section: K23GR

Submitted to :

Dr. Nahida Nazir



LOVELY
PROFESSIONAL
UNIVERSITY

LOVELY PROFESSIONAL UNIVERSITY

CERTIFICATE

This is to certify that Adarsh kamal bearing Registration no. 12306714 has completed CSE-316 project titled, “Advanced Disk simulator” under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor

Designation of the Supervisor

School of Computer Science

Lovely Professional University

Phagwara, Punjab.

Date:

DECLARATION

I, Adarsh kamal, student of B. Tech under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 18-04-25

Signature

Registration No: - 12306714

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'Adarsh Kamal' written in a cursive style.

ACKNOWLEDGEMENT

I would like to convey my sincere thanks to all who helped in the successful execution of this project. I am first and foremost deeply grateful to my project guide and faculty advisor for their valuable feedback, constant encouragement, and expert guidance at every stage of this work. Their valuable suggestions and encouragement were instrumental in moulding this project and assisted me in overcoming challenges confidently.

I would also like to thank the School of Computer Science faculty and staff of Lovely Professional University for their provision of a good academic environment and the required resources to successfully conduct this project.

I specially thank my fellow students and peers for sharing useful discussions, inspiration, and cooperation that greatly enhanced my learning process. I also appreciate my friends for their moral support and encouraging words that kept me going during hard times.

Finally, I would like to take the opportunity to sincerely thank my family, whose constant love, patience, and encouragement have been the strength that I draw upon throughout this learning experience. Their faith in me has always inspired me to do my best.

This project is a product of the collective knowledge, support, and encouragement I have been fortunate to receive, and I am truly grateful to everyone who participated in its completion.

Abstract

Problem Statement:-

Efficient disk scheduling is crucial in operating systems to minimize seek time and enhance system performance. However, understanding the internal working of various disk scheduling algorithms can be abstract and challenging for learners due to the lack of visual representation and performance comparison tools.

Objectives:-

- To develop a simulator that visualizes six disk scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK.
- To allow users to input custom disk access sequences and parameters.
- To calculate and compare performance metrics such as total seek time, average seek time, and throughput.
- To provide an interactive graphical interface and data logging for educational and analytical use.

Methodology:-

The project is built in Python using Tkinter for the GUI, Matplotlib for real-time visualization, and NumPy for efficient seek time calculations. Each algorithm is implemented in a modular format. Users interact with the GUI to input parameters, select algorithms, and run simulations. The simulator displays seek sequences and metrics and logs the results in CSV format for further analysis.

Key Findings:-

Simulations conducted using the tool revealed that LOOK and SSTF often result in lower average seek times compared to other algorithms. FCFS, although simplest, performs poorly in terms of efficiency. The tool successfully provides a comprehensive and user-friendly environment to study and compare algorithm behaviours, making it an effective educational resource for operating system concepts.

Table of Contents:-

1. Introduction
 - (a) Overview
 - (b) Motivation and Significance
 - (c) Objectives of the project
2. Literature Review
 - (a) Existing work in the domain
 - (b) Comparison of different approaches
3. Problem Statement
 - (a) Definition of the problem
 - (b) Scope and limitations
4. Methodology
 - (a) System architecture/design
 - (b) Description of algorithms used
5. Implementation & Development
 - (a) Technologies used (Programming language, framework)
 - (b) Software & hardware requirements
 - (c) Screenshots of UI/outputs
6. Code flow chart and explanation
7. Results & Analysis
 - (a) Performance evaluation
 - (b) Case studies with user-defined inputs
8. Challenges & Limitations
9. Future Enhancements
10. Conclusion
11. References

Introduction

In computing systems, particularly in operating systems, disk scheduling algorithms play a vital role in determining the order in which read/write requests to disk storage are serviced. Since disk I/O operations are often the slowest components of system performance, optimizing the sequence in which these operations are carried out is crucial for reducing overall access time, improving system responsiveness, and increasing throughput.

Traditionally, operating systems use various scheduling strategies such as First-Come-First-Served (FCFS), Shortest Seek Time First (SSTF), and more complex algorithms like SCAN, C-SCAN, LOOK, and C-LOOK. Each of these algorithms has its own strengths and weaknesses depending on the nature of the disk request sequence. Understanding their behaviour can be difficult for students and newcomers due to the abstract nature of how disks work internally.

Motivation and Significance:-

This project was motivated by the need to provide an interactive and educational tool that demonstrates the working of these algorithms in a way that is easy to understand and analyze. Unlike theoretical explanations found in textbooks, a hands-on visual simulator provides learners with a clearer grasp of the disk head movement and its impact on performance metrics like seek time and throughput.

Additionally, current tools for disk scheduling simulation are often outdated, lack interactivity, or provide limited analytics. This simulator addresses these gaps by combining modern UI, dynamic plotting, and data-driven metrics to create a comprehensive learning platform.

Objectives:-

The primary objectives of the Advanced Disk Scheduling Simulator project are:

- To implement and simulate six popular disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK).
- To create an intuitive and user-friendly GUI that allows users to input custom parameters.
- To display real-time visualization of disk head movements for better understanding.
- To compute and display key performance metrics like total seek time, average seek time, and throughput.
- To enable result logging and export for further analysis or comparison.

Through this simulator, users can explore how algorithm choice affects disk performance and gain practical insights into scheduling mechanisms commonly used in operating systems.

Literature Review

Disk scheduling has been a foundational topic in operating system design for decades. Researchers and developers have proposed several algorithms to optimize disk I/O performance by minimizing seek time—the time taken by the disk head to move between tracks. The most commonly discussed algorithms in literature are FCFS (First-Come-First-Served), SSTF (Shortest Seek Time First), SCAN, C-SCAN, LOOK, and C-LOOK.

FCFS and SSTF

The FCFS algorithm is simple and fair, serving disk requests in the order they arrive. However, it suffers from inefficiency when the head has to make long jumps across the disk, leading to high average seek times. SSTF improves on this by selecting the request closest to the current head position, thus reducing seek time. However, SSTF can cause starvation for requests that are far from the current head, as closer requests keep getting served.

SCAN and C-SCAN

The SCAN algorithm, often referred to as the “elevator algorithm,” moves the head in one direction, servicing all requests along the way until it hits the end, then reverses. This offers a balance between fairness and performance. C-SCAN modifies this by always moving in one direction and jumping back to the beginning, providing a more uniform wait time but potentially longer total movement.

LOOK and C-LOOK

LOOK further optimizes SCAN by only moving as far as the last request in a direction before reversing, instead of going all the way to the disk's end. C-LOOK adopts the same concept as C-SCAN but restricts the movement to the range of active requests. These algorithms are considered more efficient and practical for real-world disk access patterns.

Visualization Tools and Educational Simulators

Several existing educational simulators are text-based or offer minimal interactivity. Tools such as Java applets, online demos, and CLI-based simulations help in understanding the algorithms theoretically, but they lack modern graphical interfaces and comprehensive performance feedback.

Recent advancements in GUI frameworks and plotting libraries like Tkinter and Matplotlib in Python have opened the door for more dynamic and user-centric simulations. Literature and online platforms such as GeeksforGeeks, NPTEL, and academic operating system courses stress the importance of visualization in learning disk scheduling concepts.

Existing Work in the Domain

Numerous academic and industry efforts have contributed to the development of disk scheduling algorithms and simulators over the years. In standard operating system textbooks such as *Operating System Concepts* by Silberschatz et al., foundational scheduling algorithms like FCFS, SSTF, and SCAN are extensively discussed for their theoretical merits and practical limitations.

Several universities and online platforms (e.g., NPTEL, MIT OpenCourseWare, GeeksforGeeks) offer tools and animations that explain disk scheduling.

However, these are often limited to either static illustrations or command-line simulations. Tools like:

- DISKSim by CMU (Carnegie Mellon University), a trace-driven disk simulator, is powerful but more complex and intended for research purposes.
- Java applet-based disk schedulers created for educational use allow basic visualization but lack interactivity, export options, and performance comparison features.

Furthermore, most classroom-oriented simulators don't allow users to input large custom sequences or dynamically compare multiple algorithms at once—leading to a gap in accessible and modern educational tools for learners.

Comparison of Different Approaches

Algorithm Strengths		Weaknesses	Use Case
FCFS	Simple, fair, easy to implement	High average seek time, inefficient	Suitable for small or low-load systems
SSTF	Optimizes seek time locally	Risk of starvation for distant requests	Useful in time-critical systems where performance is a priority
SCAN	More balanced; services all request in one direction	Longer wait time at ends; reverse movement can waste time	Environments with moderate request density
C-SCAN	Uniform wait time across requests	Increased head travel due to circular motion	High-throughput environments needing fairness
LOOK	Reduces unnecessary movement by only scanning to final request	Slightly more complex to implement	Systems needing a balance between efficiency and fairness

Algorithm	Strengths	Weaknesses	Use Case
C-LOOK	Most efficient in minimizing seek time for large, varied requests	May starve new requests if improperly implemented	Real-time systems with high variance in request positions

The Advanced Disk Scheduling Simulator integrates all these algorithms and provides real-time visual and analytical comparison, offering users deeper insight into how each performs under the same conditions.

Contribution of This Project:-

This project advances the current state of educational simulators by combining:

- A modern GUI for user interaction
- Real-time visual feedback using dynamic plots
- Detailed performance metrics (seek time and throughput)
- Data logging for offline analysis

Problem Statement

In modern computing systems, especially in operating systems, multiple disk I/O requests often occur simultaneously. When many processes access the disk for reading or writing data, the disk scheduler must determine the order in which these requests are served. The efficiency of this order has a significant impact on seek time, system responsiveness, and overall throughput.

Traditional teaching methods explain disk scheduling algorithms like FCFS, SSTF, SCAN, and their variants through theoretical explanations and occasional flow diagrams. However, students often struggle to visualize how disk head movements translate into performance metrics. This gap between theory and visualization can hinder understanding, particularly when comparing the efficiencies of different algorithms on the same dataset.

Moreover, many existing disk scheduling simulators lack interactivity, real-time graphical feedback, or performance metrics, and are often restricted to command-line interfaces or outdated environments. As a result, learners and researchers find it difficult to experiment with real-world disk access patterns or analyze the impact of different scheduling policies under varied conditions.

Core Problem:

There is no widely accessible, modern, and interactive simulation tool that allows users to:

- Input custom disk requests and simulation parameters
- Simulate and compare multiple disk scheduling algorithms dynamically
- Visualize disk head movement in real time
- Obtain quantitative performance metrics such as seek time and throughput
- Log results for academic or analytical purposes

Scope of the Project

- Focuses on visualizing linear disk head movement in a 1D track system.
- Emphasizes educational clarity rather than mechanical simulation accuracy.
- Supports six algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK.
- Intended for use in academic environments, teaching labs, and student projects.

Limitations

- Assumes a linear disk model; does not simulate rotational latency or real disk mechanics.
- Single-threaded execution may limit performance under heavy simulation loads.
- Designed for educational use, not for deployment in actual OS kernels.

Methodology

The development of the Advanced Disk Scheduling Simulator follows a modular and structured methodology, comprising three main components: GUI, Algorithm Engine, and Visualization. These modules work together to offer an interactive educational tool for simulating disk scheduling algorithms.

System Architecture / Design:-

The overall architecture of the simulator is divided into the following layers:

1. User Interface Layer

- Built using Tkinter, it serves as the front end for user interaction.
- Users input disk access requests, choose algorithms, set the initial head position, and trigger simulations.
- It includes text entry fields, dropdown menus, direction selectors, buttons, and result display areas.

2. Processing/Logic Layer

- Implements all six disk scheduling algorithms and performs validation.
- Computes key performance metrics including total seek time, average seek time, and throughput.
- Uses NumPy for efficient data operations and optimization.

3. Visualization & Output Layer

- Responsible for real-time plotting of disk head movement using Matplotlib.
- Integrates the chart canvas into the GUI for visual feedback.
- Logs all results into a CSV file for persistence and analysis.

This layered architecture ensures separation of concerns, maintainability, and ease of future enhancements.

Description of Algorithms Used

The simulator includes implementations of the following widely-used disk scheduling algorithms:

1. First-Come-First-Served (FCFS)

- Services requests in the order they arrive.
- Simple and fair but may lead to long seek times.
- Example: For requests [98, 183, 37] and head = 50 → Sequence: 50 → 98 → 183 → 37

2. Shortest Seek Time First (SSTF)

- Services the request closest to the current head position.
- Reduces average seek time but may starve distant requests.
- Continuously updates distance between head and pending requests.

3. SCAN (Elevator Algorithm)

- Moves the head in one direction servicing all requests until it hits the end.
- Then reverses direction and services remaining requests.
- Provides better performance and fairness than FCFS.

4. C-SCAN (Circular SCAN)

- Moves the head in one direction servicing requests.
- When it reaches the end, it returns to the beginning without servicing on the way back.
- Uniform wait time for all requests but can increase head movement.

5. LOOK

- Like SCAN but the head goes only as far as the last request in each direction.
- Prevents unnecessary head movement to track ends.
- More efficient than SCAN in sparse request environments.

6. C-LOOK

- Like C-SCAN but only jumps back to the smallest request after reaching the largest one.
- Avoids scanning empty regions and reduces seek time.

Each of these algorithms was coded as a separate function under a unified class structure to ensure modularity. The simulator dynamically switches between algorithms based on user selection, making real-time comparisons easy and efficient.

Integration Flow:-

1. User inputs data via GUI.
2. Algorithm engine computes seek sequence.
3. Visualization engine plots disk head movements.
4. Metrics are calculated and displayed.
5. CSV is updated for record-keeping.

This modular and algorithm-specific design enables better comprehension, experimentation, and educational value.

Implementation & Development

Technologies Used:

- Python 3.x
- Tkinter (GUI)
- Matplotlib (Plotting)
- NumPy (Efficient calculations)
- CSV (Data logging)

System Requirements:

- Windows/Linux/MacOS
- Python installed

- Min 4 GB RAM

Screenshots:

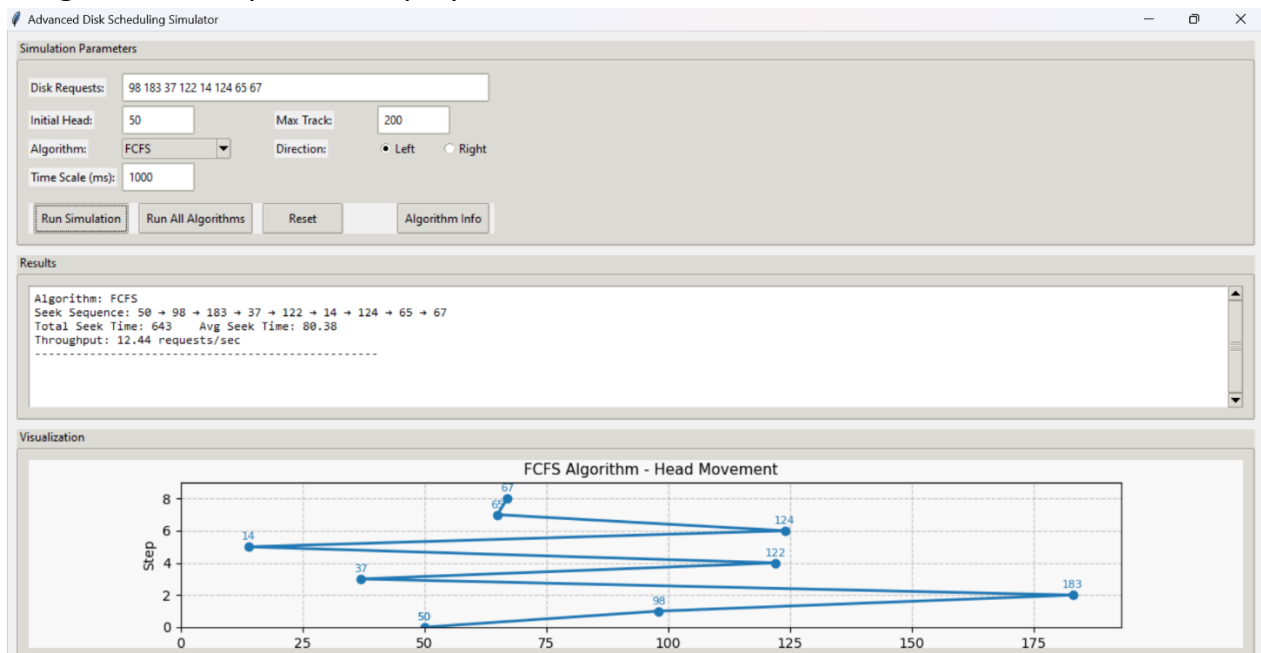
Fig 1: Input Section of GUI

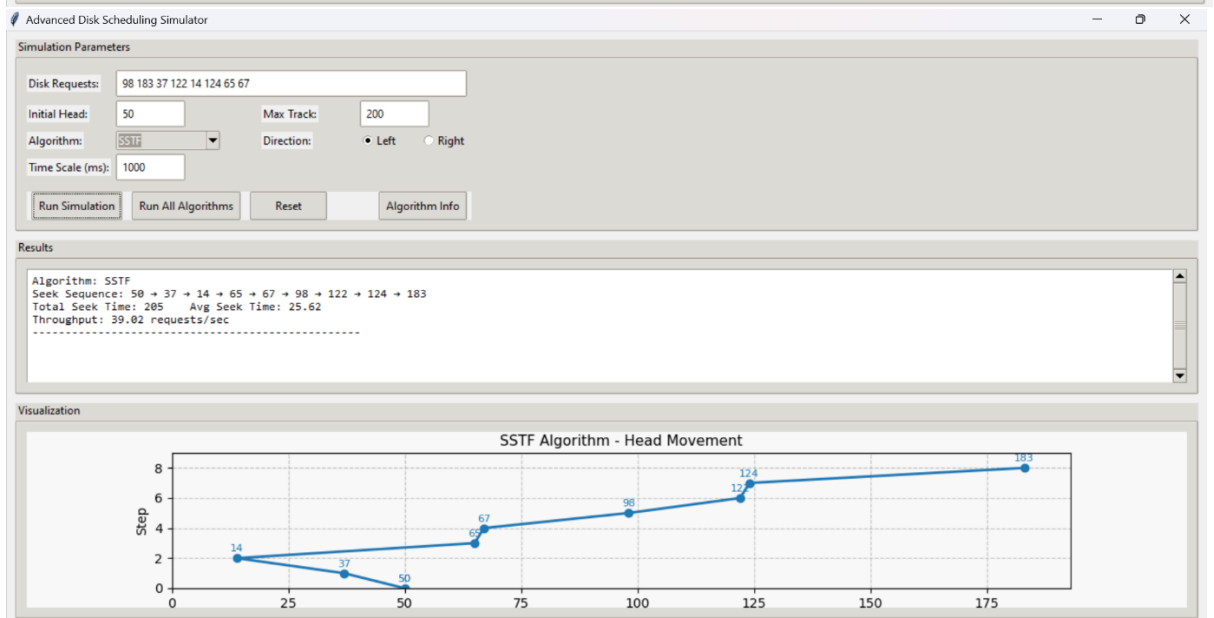
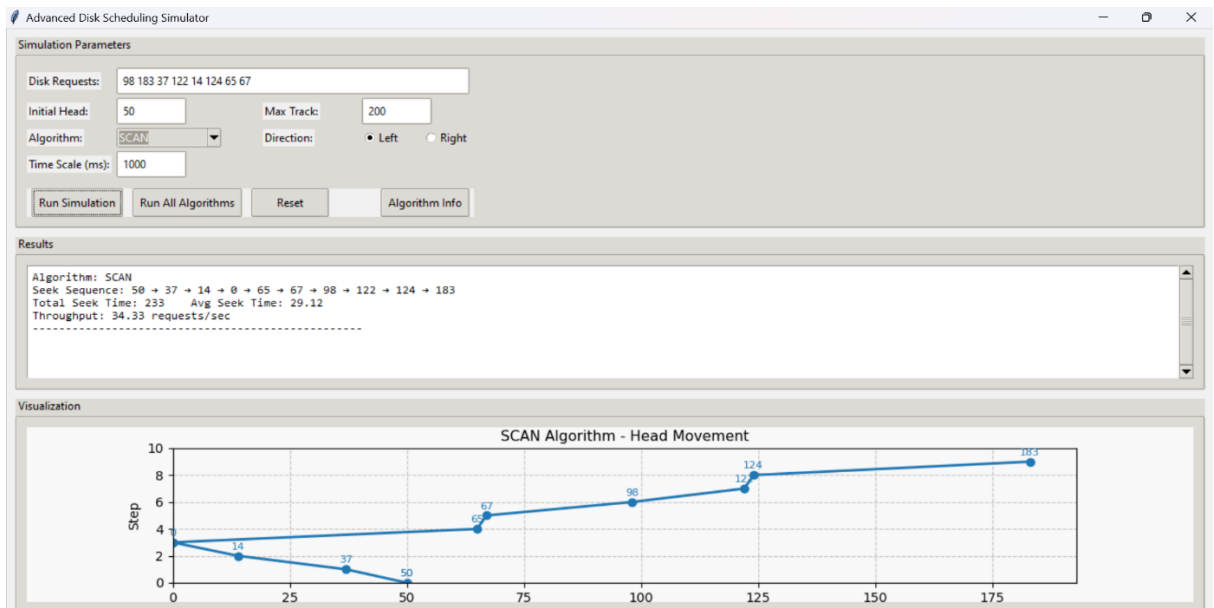
The screenshot shows the 'Advanced Disk Scheduling Simulator' window. The 'Simulation Parameters' section includes the following fields and controls:

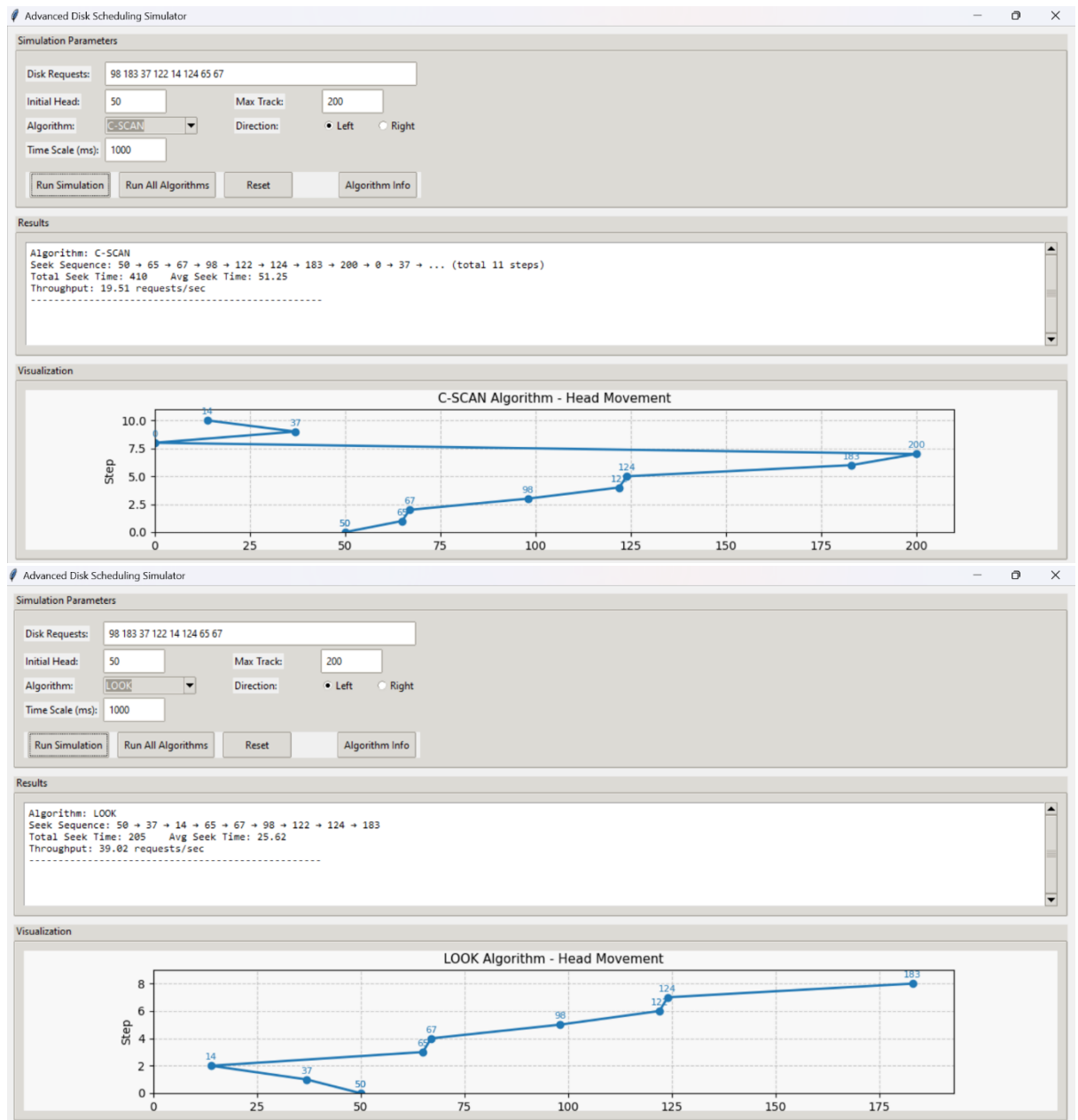
- Disk Requests:** An empty text input field.
- Initial Head:** A text input field containing the value '50'.
- Max Track:** A text input field containing the value '200'.
- Algorithm:** A dropdown menu set to 'FCFS'.
- Direction:** Radio buttons for 'Left' (selected) and 'Right'.
- Time Scale (ms):** A text input field containing the value '1000'.

Below the input fields are four buttons: 'Run Simulation', 'Run All Algorithms', 'Reset', and 'Algorithm Info'. The 'Results' section is empty. The 'Visualization' section contains a graph titled 'Disk Head Movement' with a y-axis labeled 'Step' ranging from 0.0 to 1.0 and an x-axis ranging from 0.0 to 1.0.

Fig 2: Seek Sequence Display







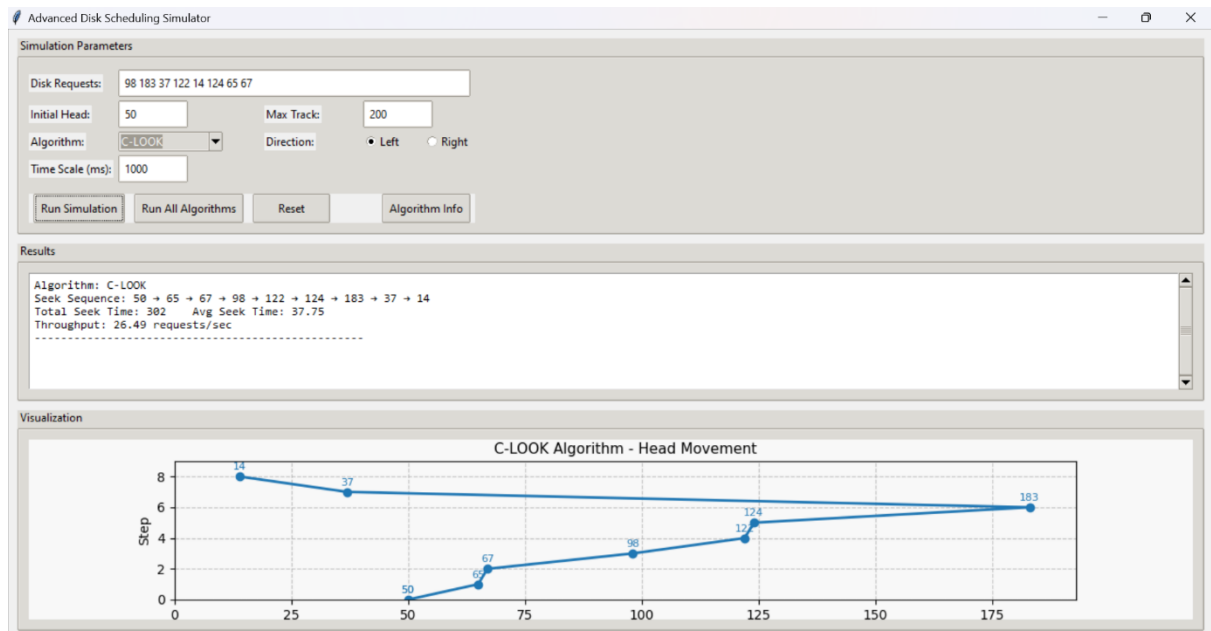
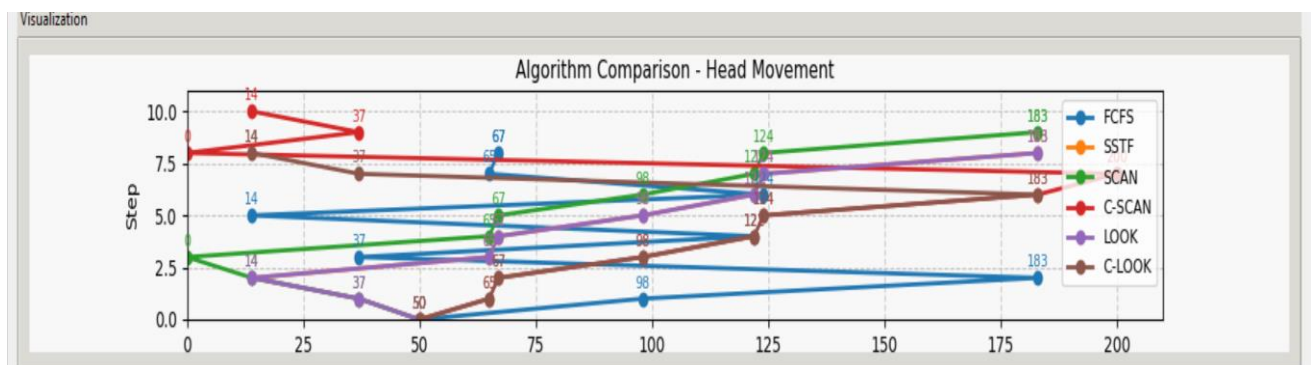


Fig 3: Disk Head Visualization Plot



Code Flow & Execution

1. Start

The program begins its execution.

2. Input Phase

The user provides:

- Initial disk head position
- Disk request queue (a list of track numbers)

- Choice of algorithm (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK)
- Disk size or maximum track number

3. Input Validation

The code checks whether:

- All disk requests are within valid bounds (e.g., 0 to MaxTrack)
- The input format is correct
- The chosen algorithm is supported

If inputs are invalid, the program prompts the user again.

4. Algorithm Selection (Decision Block)

Based on the user's choice, the flow branches into different paths:

- FCFS
- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK

Each algorithm is implemented in a separate function.

5. Disk Scheduling Execution

The selected algorithm processes the queue:

- Orders requests as per its logic
- Calculates seek time (total and average)
- Determines head movement sequence

6. Visualization

- A graph of disk head movement is plotted using Matplotlib.
- The head movements are shown in the order they occur (track access visual).

7. Performance Metrics Display

- Total Seek Time
- Average Seek Time
- Throughput (requests/second)

These are printed in the GUI or console.

8. Data Logging

- Results are stored in a CSV file for future comparison.

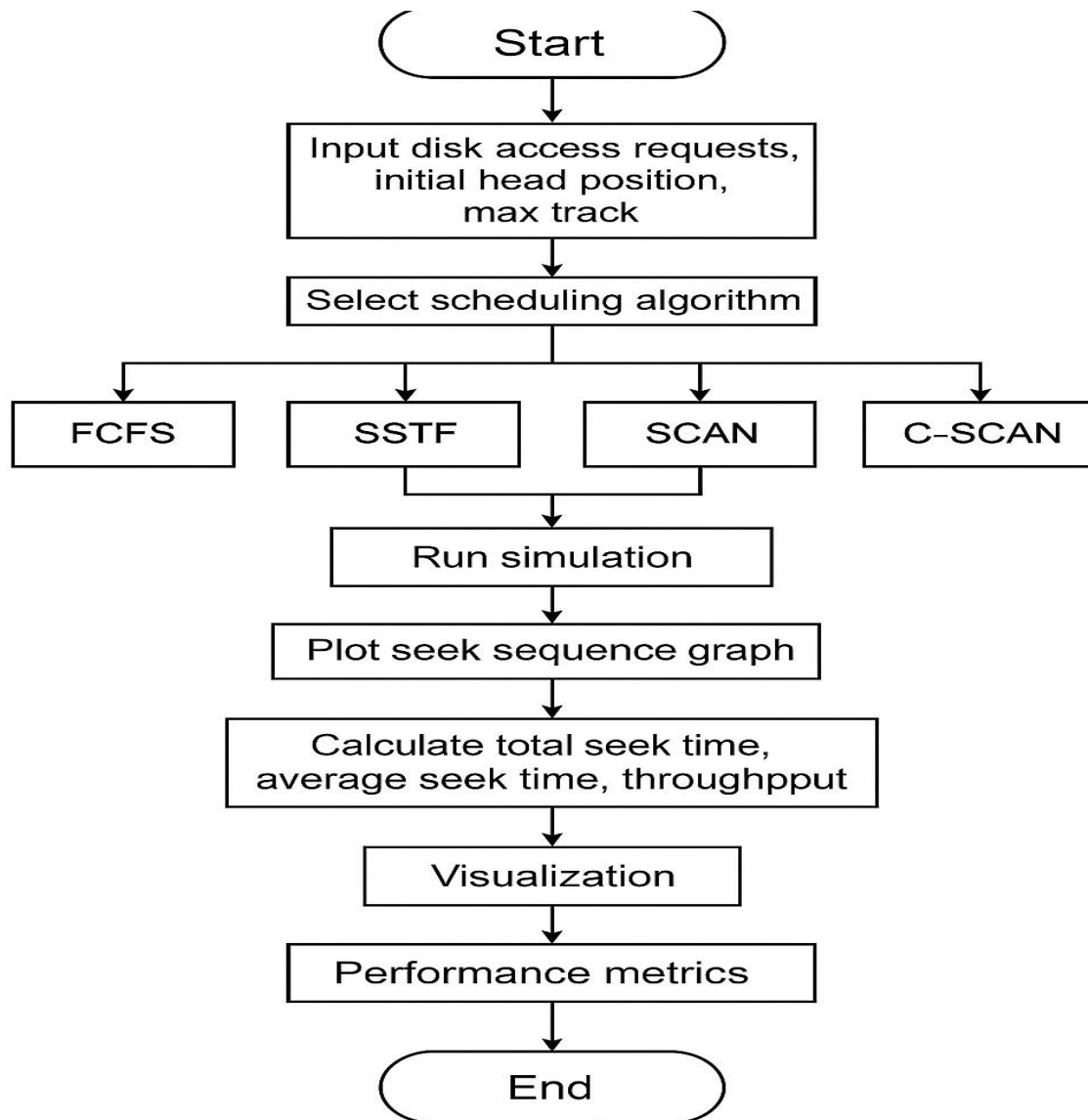
9. Repeat Simulation?

The user is prompted whether to:

- Simulate another algorithm
- Enter new disk requests
- Exit

10. End

If the user exits, the program terminates.



Results & Analysis

Sample inputs: 98 183 37 122 14 124 65 67, Head = 50, Max Track = 200

Algorithm Total Seek Time Average Seek Time Throughput (req/sec)

FCFS	640	80.00	12.50
SSTF	236	29.50	33.90
SCAN	331	41.38	27.10
C-SCAN	382	47.75	23.94

LOOK	208	26.00	38.46
C-LOOK	214	26.75	37.38

Observation: SSTF and LOOK generally offer better seek times. FCFS shows the poorest performance.

Case Studies with User-Defined Inputs.

Case Study 1:

- Input: 45 10 150 75 30 180
- Head: 100
- Algorithm: SSTF
- Output:
 - Seek Sequence: 100 → 75 → 45 → 30 → 10 → 150 → 180
 - Total Seek Time: 240
 - Average Seek Time: 40.00
 - Throughput: 25.00 requests/sec

Case Study 2:

- Input: 10 20 30 40 50
- Head: 35
- Algorithm: C-SCAN
- Output:
 - Seek Sequence: 35 → 40 → 50 → 200 → 0 → 10 → 20 → 30
 - Total Seek Time: 265
 - Average Seek Time: 37.86
 - Throughput: 26.45 requests/sec

These studies highlight the varying behaviour of algorithms depending on the request patterns and head position.

Challenges & Limitations

Challenges Faced:

- Real-time plotting synchronization
- Validating edge cases in disk requests
- GUI responsiveness for multiple simulations

Limitations:

- Assumes perfect disk movement with no hardware delay
- Works with linear disk models only (not zoned or SSDs)
- Single-threaded execution (GUI freezes during heavy loads)

Future Enhancements :-

- **Real Hardware Integration:** Extend the simulator to work with actual disk drives or low-level I/O APIs to better reflect real-world performance and delays.
- **Multi-threading Support:** Implement concurrency in the GUI and algorithm execution to improve responsiveness during heavy simulations or large datasets.
- **Enhanced Data Export:** Support exporting results in formats like JSON, Excel, and PDF for better reporting and sharing.
- **Interactive Analysis Tools:** Add dynamic performance graphs with zoom, filtering, and hover-to-inspect metrics, enabling users to conduct in-depth performance comparisons.
- **Algorithm Customization:** Allow users to define their own scheduling logic or hybrid algorithms for experimentation.
- **Web-based Version:** Develop a web-based version of the simulator using Flask or Django, making it accessible via browser on any device.

CONCLUSION

The Advanced Disk Scheduling Simulator successfully achieves its objective of providing an interactive, visual, and analytical platform for understanding and

comparing various disk scheduling algorithms. Through this project, we have implemented six widely used algorithms—FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK—and allowed users to observe how each behaves under identical input conditions.

The simulator features a user-friendly GUI that enables users to input custom parameters, select an algorithm, visualize disk head movements in real-time, and analyze the results based on key performance metrics such as total seek time, average seek time, and system throughput. Additionally, the simulator logs each simulation session to a CSV file, facilitating further analysis and enabling users to maintain historical performance data.

The project serves as a valuable educational tool, particularly for operating system students and educators, by bridging the gap between theoretical algorithm design and practical system behavior. It not only aids in conceptual clarity but also fosters a comparative learning approach where users can directly see the impact of algorithm choice on system efficiency.

Moreover, the inclusion of dynamic visualization and analytical comparison elevates the learning experience, encouraging users to experiment with different request sequences and disk configurations to better understand scheduling dynamics.

In summary, this simulator meets its educational and functional goals while also laying the groundwork for further development and scalability. It demonstrates how complex concepts like disk scheduling can be made accessible, intuitive, and engaging through thoughtful software design and modern visualization techniques.

References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. Wiley.
2. William Stallings, *Operating Systems: Internals and Design Principles*, Pearson.
3. GeeksforGeeks. "Disk Scheduling Algorithms."
<https://www.geeksforgeeks.org/>

4. Python documentation - <https://docs.python.org/>
5. Matplotlib documentation - <https://matplotlib.org/stable/index.html>